EXP.NO: 4

SystemCalls: fork(), exec(), getpid(), opendir(), readdir()

DATE:03/02/2026

AIM:

Tostudy and implement Linux system calls — fork() for process creation, exec() for process image replacement, getpid() for retrieving process IDs, and opendir()/readdir() for directory traversal — using C programming.

ALGORITHM:

1. Start the experiment.
2. Write a C program and include required headers: unistd.h, sys/types.h, sys/wait.h, dirent.h. 3. Use getpid() to print the current process ID.
4. Call fork() to create a child process; the return value distinguishes parent and child.
5. In the child process (pid == 0), use execvp() to replace the process image with ls -l.
6. In the parent process (pid > 0), call wait(NULL) to wait for the child to finish.
7. Use opendir(".") to open the current directory stream.
8. Use readdir() in a loop to read each directory entry.
9. Close the directory stream with closedir().
10. Compile with gcc and execute the binary.
11. Stop the experiment.

CODE:

```
#include     <stdio.h>
#include    <stdlib.h>
#include    <unistd.h>
#include  <sys/types.h>
#include   <sys/wait.h>
#include <dirent.h>

int main() {
    pid_t pid;

    /* getpid() - current process ID */
    printf("Parent PID: %d\n", getpid());

    /* fork() - create child process */
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }
    elseif(pid== 0) {
        /* ---Child process --- */
        printf("Child PID: %d, Parent PID: %d\n",
```

```
                getpid(), getppid());

        /* exec() - replace child image with 'ls -l' */
        char *args[] = {"ls", "-l", NULL};
        execvp("ls", args);

        perror("exec failed");   /* reached only if exec fails */
        exit(1);
    }
    else {
    /*     --- Parent process --- */
    wait(NULL);                  /* wait for child */
        printf("Child process completed.\n");

        /* opendir / readdir - list directory */
        DIR *dir;
        struct dirent *entry;

        dir = opendir(".");
        if (dir == NULL) { perror("opendir failed"); exit(1); }

        printf("\nDirectory contents:\n");
        while((entry = readdir(dir)) != NULL)
            printf(" %s\n", entry->d_name);

        closedir(dir);
    }
    return 0;
}
```

OUTPUT:

```
[hp@localhost ~]$ gcc syscalls.c -o syscalls
[hp@localhost ~]$ ./syscalls
Parent PID: 4521

total 28
-rwxr-xr-x 1 hp hp 8432 Feb 10 09:15 syscalls
-rw-r--r--  1  hp  hp 892 Feb 10 09:14 syscalls.c
-rw-r--r-- 1 hp hp  134 Feb 03 10:20 students.txt

Child PID: 4522, Parent PID: 4521
Child process completed.

Directory contents:
  .
 ..
 syscalls
syscalls.c
students.txt
```

RESULT:

Thus,Linux system calls fork(), exec(), getpid(), opendir(), and readdir() were successfully implemented and demonstrated using C programs in the Linux environment.